```
#include <Arduino.h>
#include <FS.h>
#include <SD_MMC.h>

HardwareSerial BridgeSerial(1);

#define BRIDGE_RX 12
#define BRIDGE_TX 13

#define PENDING_FILE "/trace_pending.txt"

void setup() {

 Serial.begin(115200);

 BridgeSerial.begin(
  9600,
  SERIAL_8N1,
  BRIDGE_RX,
  BRIDGE_TX
 );

 delay(1000);

 Serial.println();
 Serial.println("================================");
 Serial.println("DFR0602 OFFLINE STORAGE BRIDGE");
 Serial.println("================================");
```

```
 if (!SD_MMC.begin("/sdcard", true)) {

  Serial.println("SD CARD INIT FAILED");

  BridgeSerial.println("SD_ERROR");

  return;
 }

 uint8_t cardType = SD_MMC.cardType();

 if (cardType == CARD_NONE) {

  Serial.println("NO SD CARD");

  BridgeSerial.println("SD_ERROR");

  return;
 }

 Serial.println("SD CARD INITIALIZED");
 Serial.println("OFFLINE STORAGE BRIDGE READY");

 BridgeSerial.println("STORAGE_BRIDGE_READY");
}

// ==================================================
```

```
// SEND ONE RECORD AND WAIT FOR ACK
// ================================================

bool sendRecordWithACK(String line) {

 BridgeSerial.println(line);

 Serial.print("SENT RECORD: ");
 Serial.println(line);

 unsigned long startTime = millis();

 while (millis() - startTime < 5000) {

  if (BridgeSerial.available()) {

   String response =
    BridgeSerial.readStringUntil('\n');

   response.trim();

   Serial.print("ESP RESPONSE: ");
   Serial.println(response);

   if (response == "ACK") {

    Serial.println("ACK RECEIVED");
```

```
      return true;
    }

    if (response == "NACK") {

      Serial.println("NACK RECEIVED");

      return false;
    }
  }

  delay(2);
 }

 Serial.println("ACK TIMEOUT");

 return false;
}

// ================================================
// MAIN LOOP
// ================================================

void loop() {

 if (!BridgeSerial.available()) {
  return;
 }
```

```
String command =
  BridgeSerial.readStringUntil('\n');

command.trim();

Serial.print("UART COMMAND: ");
Serial.println(command);

// ================================================
// PING
// ================================================

if (command == "PING") {

  BridgeSerial.println("PONG");
}

// ================================================
// STATUS
// ================================================

else if (command == "STATUS") {

  BridgeSerial.println("SD_OK");
}

// ================================================
```

```
// WRITE
// ================================================

else if (command.startsWith("WRITE,")) {

 String data =
  command.substring(6);

 File file =
  SD_MMC.open(
   PENDING_FILE,
   FILE_APPEND
  );

 if (!file) {

  Serial.println("WRITE ERROR");

  BridgeSerial.println("WRITE_ERROR");

  return;
 }

 file.println(data);

 file.close();

 Serial.println("SD WRITE SUCCESS");
```

```
 BridgeSerial.println("WRITE_OK");
}

// ================================================
// READ
// ================================================

else if (command == "READ") {

 File file =
  SD_MMC.open(
   PENDING_FILE,
   FILE_READ
  );

 if (!file) {

  BridgeSerial.println("READ_EMPTY");

  return;
 }

 if (file.size() == 0) {

  file.close();

  BridgeSerial.println("READ_EMPTY");
```

```
 return;
}

// ------------------------------------------------
// Tell ESP8266 that reading is starting
// ------------------------------------------------

BridgeSerial.println("READ_START");

delay(100);

bool transferOK = true;

// ------------------------------------------------
// Send records ONE BY ONE
// ------------------------------------------------

while (file.available()) {

 String line =
  file.readStringUntil('\n');

 line.trim();

 if (line.length() == 0) {
  continue;
 }
```

```
  Serial.println();
  Serial.println("--------------------------------");
  Serial.println("SENDING STORED RECORD");
  Serial.println(line);

  // ---------------------------------------------
  // Send record and wait for ACK
  // ---------------------------------------------

  if (!sendRecordWithACK(line)) {

    Serial.println(
      "TRANSFER FAILED"
    );

    transferOK = false;

    break;
  }

  // ---------------------------------------------
  // Extra safety delay
  // ---------------------------------------------

  delay(50);
}
```

```
  file.close();

  // ------------------------------------------------
  // Only tell ESP8266 READ_END if transfer
  // completed successfully
  // ------------------------------------------------

  if (transferOK) {

    delay(100);

    BridgeSerial.println("READ_END");

    Serial.println(
      "ALL RECORDS SENT SUCCESSFULLY"
    );
  }

  else {

    Serial.println(
      "READ TRANSFER FAILED"
    );
  }
}

// ================================================
// CLEAR
```

```
// ================================================

else if (command == "CLEAR") {

 if (SD_MMC.exists(PENDING_FILE)) {

  if (SD_MMC.remove(PENDING_FILE)) {

   Serial.println(
    "SD PENDING FILE CLEARED"
   );

   BridgeSerial.println("CLEAR_OK");
  }

  else {

   Serial.println(
    "CLEAR ERROR"
   );

   BridgeSerial.println(
    "CLEAR_ERROR"
   );
  }
 }

 else {
```

```
    BridgeSerial.println("CLEAR_OK");
   }
  }

  // ====================================================
  // UNKNOWN COMMAND
  // ====================================================

  else {

   BridgeSerial.println(
    "UNKNOWN_COMMAND"
   );
  }
 }
```